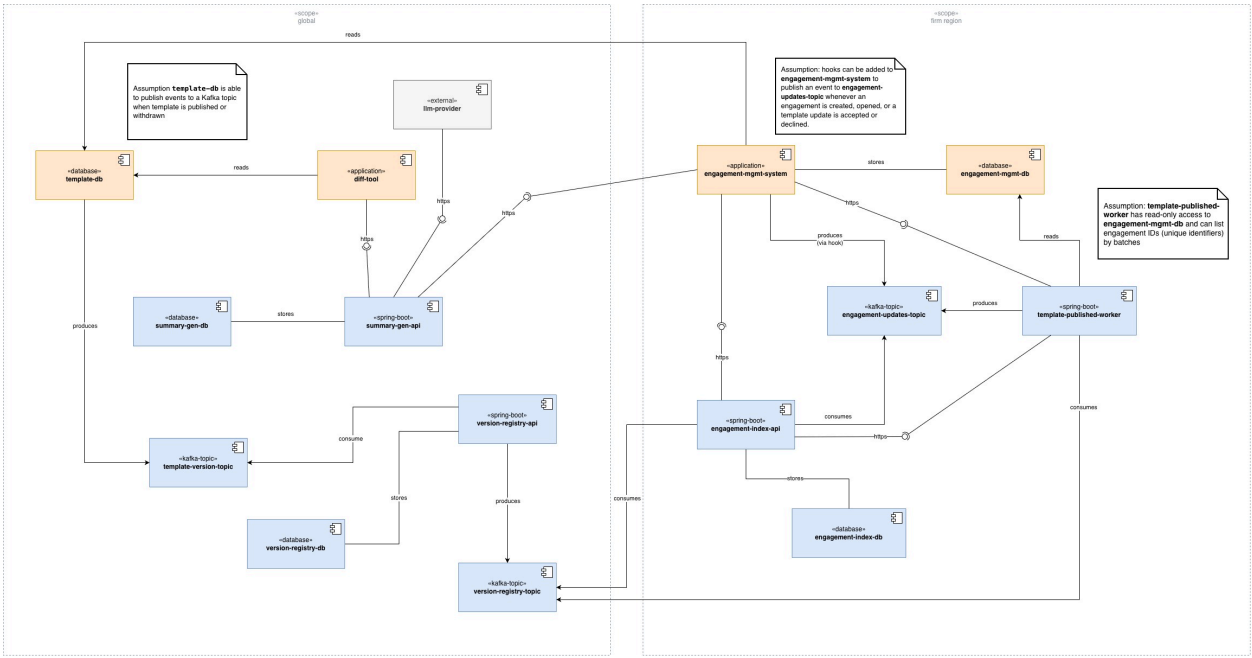
Home Task

Introduction

This document describes the architecture of the solution that lets firms see which engagement files have pending template updates, along with a human-readable summary of the inbound changes to support the accept or decline decision.

Development View



The solution is split into global services, which handle template content shared by all firms, and regional services, which hold firm data and are deployed in each region (EU, Canada, rest of the world). Components in orange already exist; components in blue are new.

Global

- `template-db`: stores every version of every product template. Publishes an event via hooks whenever the content team publishes or withdraws a version (see the assumption in the diagram).

Direction	Topic	Message key	Events
Produces (via hooks)	`template-version-topic`	`templateId`	`TemplateVersionPublished`, `TemplateVersionWithdrawn`

- `diff-tool`: exposes a REST endpoint that, given a template ID and two versions, returns the JSON diff between them.
- `llm-provider`: turns a JSON diff into a human-readable summary.
- `version-registry-api`: tracks the current version of each template per market and how versions relate to each other.

Direction	Topic	Message key	Events
Consumes	`template-version-topic`	`templateId`	`TemplateVersionPublished`, `TemplateVersionWithdrawn`
Produces	`version-registry-topic`	`templateId`	`CurrentVersionChanged`

- `version-registry-db`: stores the version graph and the current version of each template per market.
- `summary-gen-api`: exposes a REST endpoint that, given a template ID and two versions, returns the human-readable summary of what changed. Each summary is generated once, using `diff-tool` and `llm-provider`, and reused afterwards.
- `summary-gen-db`: stores summaries immutably, with the diff hash, model and prompt version used, so they can be defended in a later review.

Firm region

- `engagement-mgmt-system`: creates and loads engagements, processes accept/decline decisions, and publishes events via hooks (see the assumption in the diagram). Its UI shows pending updates and their summaries. Exposes an API that opens an engagement on request (~1 minute per engagement), used by `template-published-worker`.

Direction	Topic	Message key	Events
Produces (via hooks)	`engagement-updates-topic`	`engagementId`	`EngagementCreated`, `EngagementOpened`,

Direction	Topic	Message key	Events
			`UpdateAccepted`, `UpdateDeclined`

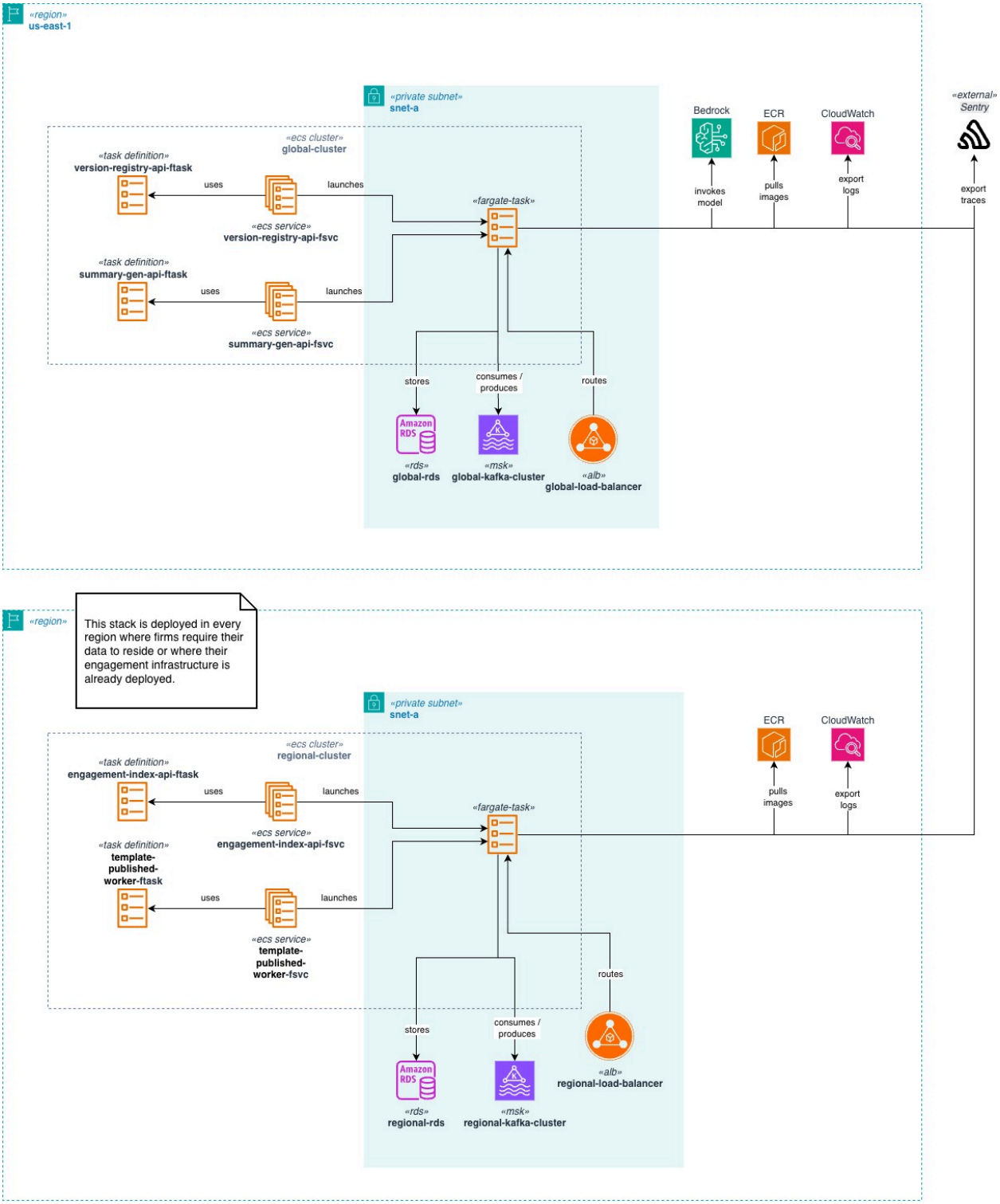
- `engagement-mgmt-db`: stores the engagements of each firm. Only their unique IDs can be read directly (see the assumption in the diagram); their content is not queryable and can only be read by loading the engagement through `engagement-mgmt-system`.
- `engagement-index-api`: keeps a log in its database of every change to an engagement since its creation (immutability), and marks the engagement as pending update when its template version is behind the current one and the user hasn't declined it. Serves pending updates to the UI, and the IDs of the engagements it already knows to `template-published-worker`, both via a REST API.

Direction	Topic	Message key	Events
Consumes	`engagement-updates-topic`	`engagementId`	`EngagementCreated`, `EngagementOpened`, `UpdateAccepted`, `UpdateDeclined`, `TemplateUpdateAvailable`
Consumes	`version-registry-topic`	`templateId`	`CurrentVersionChanged`

- `engagement-index-db`: stores the log of changes of each engagement and its pending state.
- `template-published-worker`: when the current version of a template changes, makes sure every engagement is evaluated against it. Lists engagement IDs from `engagement-mgmt-db` and asks `engagement-index-api` which ones are already known. Known engagements get a `TemplateUpdateAvailable` event carrying the current version; unknown ones are opened through `engagement-mgmt-system`, within its capacity limit.

Direction	Topic	Message key	Events
Consumes	`version-registry-topic`	`templateId`	`CurrentVersionChanged`
Produces	`engagement-updates-topic`	`engagementId`	`TemplateUpdateAvailable`

Deployment View



All new services run on Amazon ECS with Fargate, in private subnets. The global stack runs in us-east-1. The firm region stack is deployed in every region where firms require their data to reside or where their engagement infrastructure is already deployed. In each region, a single Fargate task represents the tasks of every service in that region.

Global (us-east-1)

- `global-cluster`: ECS cluster running one service per microservice, each with its own task definition:

ECS service	Task definition	Runs
`version-registry-api-fsvc`	`version-registry-api-ftask`	`version-registry-api`
`summary-gen-api-fsvc`	`summary-gen-api-ftask`	`summary-gen-api`

- `global-rds`: a single RDS instance with one schema per service (`version_registry`, `summary_gen`). Each service has its own database user restricted to its schema, and `summary-gen-api` can only insert and read summaries.
- `global-kafka-cluster`: MSK cluster hosting `template-version-topic` and `version-registry-topic`. Used by `version-registry-api`.
- `global-load-balancer`: internal ALB that routes requests to `summary-gen-api`.
- Bedrock: LLM provider invoked by `summary-gen-api`.

Firm region

- `regional-cluster`: ECS cluster running one service per microservice, each with its own task definition:

ECS service	Task definition	Runs
`engagement-index-api-fsvc`	`engagement-index-api-ftask`	`engagement-index-api`
`template-published-worker-fsvc`	`template-published-worker-ftask`	`template-published-worker`

- `regional-rds`: RDS instance hosting `engagement-index-db`.

- `regional-kafka-cluster`: MSK cluster hosting `engagement-updates-topic` and a replica of `version-registry-topic`, copied from `global-kafka-cluster` with MSK Replicator.
- `regional-load-balancer`: internal ALB that routes requests from the `engagement-mgmt-system` UI to `engagement-index-api`. The UI reaches `global-load-balancer` for summaries.

Shared services

- ECR: stores the images of every service and replicates them to every region.
- CloudWatch: receives the logs of every task.
- Sentry: receives the traces of every task. Traces carry only technical data (IDs, timings, errors), never engagement content.

Reliable Delivery and Failure Recovery

- Producers publish with `acks=all`, so an event is confirmed only once Kafka has stored it on all in-sync replicas.
- Consumers use at-least-once delivery: they commit their offset only after processing an event. If a consumer fails, it resumes from its last committed offset and no event is lost.
- Events with the same message key (`templateId` or `engagementId`) always land in the same partition, so consumers read them in the order they were produced.

Cost Estimate

Global (us-east-1)

Resource	USD/month
Fargate (4 tasks)	72
RDS	51
MSK	102

Resource	USD/month
ALB	20
NAT Gateway	35
Bedrock	40
Subtotal	**320**

Per firm region (×3)

Resource	USD/month
Fargate (4 tasks)	72
RDS	51
MSK	102
ALB	20
NAT Gateway	35
Subtotal	**280 × 3 = 840**

Shared

Resource	USD/month
MSK Replicator (×3)	657
Sentry (US and EU organizations)	52
ECR and CloudWatch	~10
Subtotal	**~719**

Total: ~$1,880/month (~$1,960 including the EU and Canada price premium).